BOAS PRÁTICAS DO SCRIPT

`limpa.sh` : dezenove linhas escritas numa tarde de 2023, sem shebang, sem comentário, com `777` num diretório do grupo `devs` — e rodando às 4h no crontab do root de três servidores. Este capítulo é sobre não escrever esse arquivo.

057
GUIAS LINUX

O MOLDE — COMECE TODO SCRIPT DAQUI

comprime — o esqueleto que vale adotar no primeiro dia

```
#!/usr/bin/env bash
# comprime - resumo em uma linha | Uso: comprime [-v] [-n] ARQUIVO...
# Saidas: 0 ok | 1 erro | 2 uso incorreto | 3 dependencia ausente | Versao: 1.2.0
set -euo pipefail
readonly PROGRAMA="${0##*/}"        # nome sem chamar basename
readonly AQUI="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
DESTINO="${DESTINO:-/var/backups}"  # ajustavel pelo ambiente, padrao seguro
erro() { printf '%s: %s\n' "$PROGRAMA" "$*" >&2; }
main() {
  command -v gzip >/dev/null || { erro "gzip ausente"; exit 3; }
  (( $# )) || { uso >&2; exit 2; }
  for a in "$@"; do
    (( SIMULAR )) && { printf 'SIMULACAO: gzip -- %q\n' "$a"; continue; }
    gzip -- "$a"                    # o -- protege nome iniciado por hifen
  done
}
main "$@"
```

`main "$@"`
O shell lê o arquivo *enquanto* executa: com tudo em funções e a chamada no fim, ele precisa ter lido o arquivo **inteiro** antes de rodar qualquer coisa — imune a edição em execução e a *download* truncado.

`$BASH_SOURCE`
O diretório do **script**, não o seu: `$PWD` depende de onde o usuário estava e `$0` falha sob `source`. O `pwd -P` ainda resolve os links.

`mktemp` + `trap`
`tmp="$(mktemp -d)"` e `trap 'rm -rf -- "$tmp"' EXIT INT TERM`: limpa na saída normal, no erro sob `set -e` e no sinal. Execução única: `flock -n 9`.

`-n` · idempotência
Todo script destrutivo merece **simulação barata de digitar**, e o `%q` mostra o nome escapado como o shell o vê. Rodar duas vezes não pode estragar: `mkdir -p`, `rm -f`, `ln -sfn`, `install -d`.

QUEM PODE REESCREVER ? · E EM QUE CAMADA VOCÊ ESTÁ

✗ VULNERÁVEL
`-rwxrwxr-x root:devs /opt/limpa.sh`

1. O *crontab* do root executa o script às 4h.
2. Qualquer membro de `devs` reescreve o arquivo.
3. Às 4h01, o que esse membro escreveu roda **como root**.

✓ CORRETO
`-rwxr-xr-x root:root /usr/local/bin/limpa`

1. Instalado com `sudo install -o root -g root -m 755`.
2. Só o root grava; os demais leem e executam.
3. Às 4h01, roda o que o root aprovou.

↑ MAIS PORTÁTIL — MAIS RECURSOS ↓

`#!/bin/sh` — POSIX puro
dash, busybox ash, Alpine, *initramfs*, contêiner mínimo. **Sem** `[[ ]]`, arrays, `local`, `${var,,}`, `$RANDOM`, `{1..10}`.

`#!/usr/bin/env bash` — Bash 3
Linux, macOS (que ainda traz o 3.2), BSD com Bash. Ganha `[[ ]]`, `=~`, arrays indexados, `local`, `(( ))`. **É a camada certa para administração de Linux.**

Bash 4+
`declare -A`, `${var,,}`, `mapfile`, `globstar`. Ausente no macOS de fábrica.

+ utilitários GNU
`sed -i`, `date -d`, `readlink -f`, `sort -V`, `find -printf`. Sintaxe diferente no BSD/macOS, ausentes no busybox.

Ao descer para POSIX: `[[ $a == b* ]]` → `case` · `(( n > 10 ))` → `[ "$n" -gt 10 ]` · `arr=(a b c)` → `set -- a b c` · `source` → `.` · `echo -e` → `printf`. Verifique com `checkbashisms`, `shellcheck -s sh` e uma execução real sob `dash script.sh`.

SUID em script de shell não funciona — e a tentação de contornar isso é o buraco. O kernel **ignora** o bit SUID em arquivo com *shebang* (seria explorável por corrida entre a leitura do *shebang* e a abertura pelo interpretador). A saída legítima é uma regra específica no `sudoers`, criada com `visudo -f /etc/sudoers.d/…` — e aí a permissão do arquivo vira o ponto crítico: com um `NOPASSWD` apontando para um script **gravável pelo grupo**, quem grava tem root sem senha. Evite curinga na regra (`/usr/local/bin/*`) e `sudo` de interpretador — `sudo python3`, `perl`, `awk`, `vim` equivalem a `sudo su`. `chmod 777` **não tem caso de uso legítimo.** Audite com `namei -l /usr/local/bin/cmd` (o caminho *inteiro*) e `sudo find /usr/local/bin /opt -type f -perm -o+w -ls`: a resposta esperada é vazia. A regra em uma frase — **tudo que um processo privilegiado executa ou carrega deve ser gravável apenas por quem já tem esse privilégio**: o arquivo, o diretório que o contém, todos os diretórios acima, o `.conf` que ele lê e a biblioteca que ele carrega com `source`.

RESUMO RÁPIDO

- ✓ **Sempre o mesmo molde:** *shebang* · comentário com uso e códigos de saída · `set -euo pipefail` · constantes `readonly` · funções de mensagem · `main "$@"` na última linha.
- ✓ **A pergunta da permissão é "quem pode reescrever?"**, não "quem executa". `install -o root -g root -m 755` em `/usr/local/bin` — diretório que nenhum pacote toca.
- ✓ **Escolha a camada de portabilidade uma vez**, declare-a no *shebang* e verifique com ferramenta. "Portabilidade que não se testa é portabilidade imaginária."
- ✓ `shellcheck` **não é opcional** em código que outra pessoa executa. Silencie por *ocorrência* e com justificativa (`# shellcheck disable=SC2016`), nunca por arquivo — e ponha num *hook* de pré-commit.
- ✓ Os campeões: **SC2086** aspas · **SC2046** `$( )` sem aspas · **SC2006** crase · **SC2045** `ls` em laço · **SC2164** `cd` sem tratar falha · **SC2181** testar `$?` em vez do comando no `if`.
- ✓ **Não use** `IFS=$'\n\t'` como "modo estrito": muda comportamento global para compensar defeito local, e quebra `read`, `set --` e código de terceiros. Aspas em toda expansão resolve sem efeito colateral.

NO KALI

Num *rolling*, script é código vivo. Uma ferramenta muda de opção ou de formato de saída entre duas execuções do mesmo script — as três defesas: `command -v` antes de usar, **saída estruturada** (`-oX`, `--json`) em vez de analisar texto de tela, e uma linha de versão para saber com o que ele foi testado. **Regra `NOPASSWD` apontando para script é a escalada mais encontrada em máquina de laboratório**: se você monta ambiente de treino, está criando o vetor de propósito; se administra a sua máquina, rode `sudo -l` e trate cada linha que aponte para arquivo gravável por não-root como root disponível. **Script de live USB ou de *initramfs* está na camada cinza**: ali o shell é o `busybox ash` e `[[ ]]` e arrays simplesmente não existem — escreva em POSIX desde o início e teste com `dash script.sh`, porque descobrir isso durante uma recuperação real é o pior momento possível. E **automação de ferramenta de segurança merece registro e simulação por padrão**: valide o alvo contra a lista do escopo autorizado, exiba o que fará, exija confirmação (ou um `--sim` explícito) e registre horário, comando e código de saída. Serve para depurar e serve para demonstrar depois que o trabalho ficou dentro do contratado — uso não autorizado, ainda que por erro de script, é crime pela Lei 12.737/2012.